

# Perceptor.AI

**What breaks if you release this change?**

Predictive Engineering Review & Code Evaluation for Proactive Threat Observation & Risk

# Releases fail where a diff cannot see

## The chain

Authentication is removed from one handler. Eight callers still compile. The tests that would catch it were deleted in the same change.

## Reviewers skim

Hundreds of PRs merge weekly. Fatigue and LGTM rubber-stamping let bugs and leaked secrets reach production.

## Tools are siloed

Static tools, tests and CI report separately. Nothing answers: is this release safe, and why?

Knight Capital (2012): \$460M+ lost in 45 minutes. CrowdStrike (2024): one bad update crashed millions of PCs.  
The gap: no tool joins code analysis, security and repository context into one explainable release decision.

# Not 'is this code okay?' but 'what breaks if we ship it?'

## Input

Paste a GitHub URL or upload a zip.

## Analysis

Parse, diff, rules, blast radius, optional AI review.

## Decision

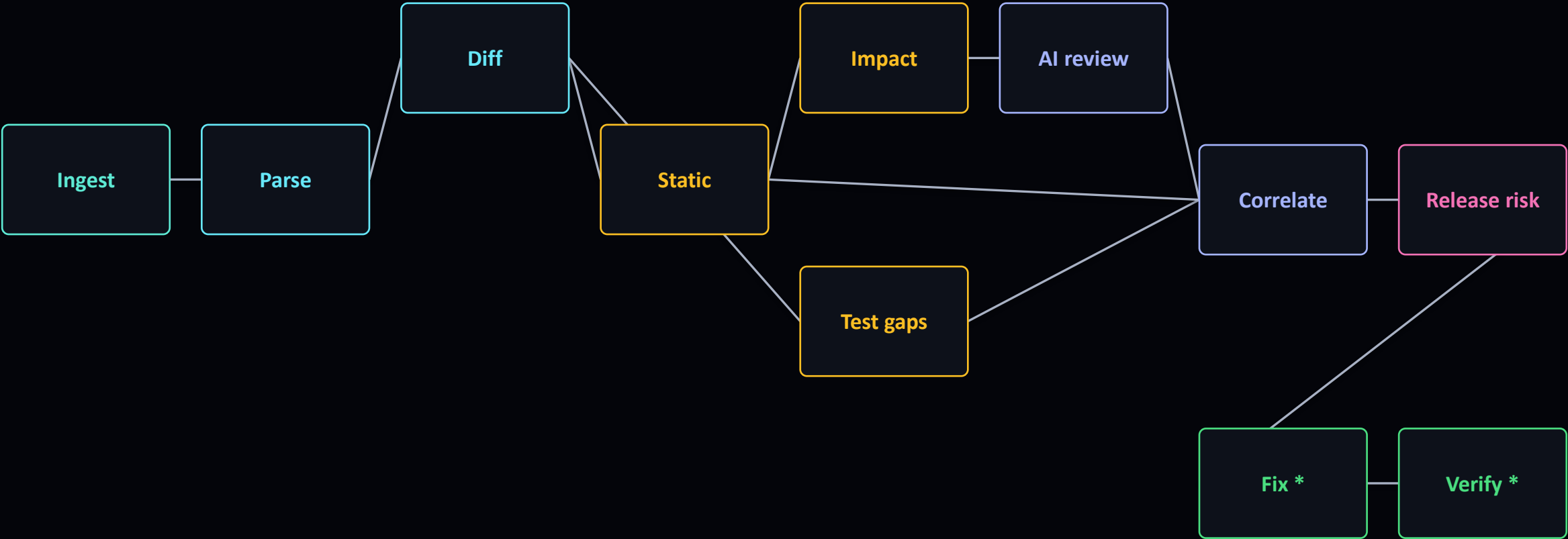
NOT READY / READY WITH WARNINGS / READY, each with reasons.

## Proof

Apply repairs, compile, run tests, rescore.

- Every finding keeps its line, evidence, impact and fix.
- Works with no model key. The AI only reviews evidence the engine already found.
- Exports Markdown and SARIF 2.1.0 for existing security tooling.

# Evidence first, AI second



■ Source      ■ Understand      ■ Detect      ■ Reason      ■ Decide      ■ Close the loop

\* optional stage. Stages, dependencies and layout are read from config/pipeline.yaml.

# 14 detectors across security, regression and quality

## Regression (3)

- Lookup used without a missing-value check
- Input validation removed
- Response status changed on an endpoint

## Quality (2)

- Changed function is hard to follow
- Changed function is very long

## Performance (1)

- Query executed inside a loop

## Deployment (2)

- Dependency version moved backwards
- Unsafe runtime configuration

## Testing (1)

- Regression test removed for a changed function

## Security (5)

- SQL built with string interpolation
- Secret material stored in source
- Command built from concatenated input
- Untrusted text written as HTML
- Authentication check removed from a handler

Rules are YAML under rules/. Add a detector without changing code.

# See the blast radius, not just the diff

- Builds a symbol and call graph (tree-sitter + Python AST + NetworkX).
- Maps each changed symbol to callers, callees and affected endpoints.
- Builds a failure chain from the change up to the endpoint users hit.
- Simulate release walks that chain step by step.
- Flags regression tests removed while the code they covered was changing.

# An explainable score, not a black box



## Points per finding

Critical: 40  
High: 25  
Medium: 12  
Low: 5  
Info: 2

Each dimension is  $100 \times (1 - e^{-(\text{points} / 100)})$ : it never caps, so more findings always score higher. Bands: Low 0-25, Medium 26-50, High 51-75, Critical above.

Decision is separate from the band: any Critical finding, or a High finding in a blocking category, means NOT READY.

# It proposes a fix, then proves it holds

## 1. Repair

Parameterized SQL, secrets moved to the environment, restored checks, tests and pins.

## 2. Verify

Work on a copy. Run compileall and pytest with a timeout. The original tree is untouched.

## 3. Rescore

Before and after on one screen. Blocking findings cleared means the status improves.

Query-in-loop is deliberately not auto-fixed. It is not a blocker, so the repaired tree ends at READY WITH WARNINGS.

# 10 of 10 planted issues found, 0 missed

## Detection

Planted 10  
Detected 10  
Missed 0  
Extra 0

## Decision

NOT READY  
Score 43  
Analysis 0.043s

## Speed

51 files: 0.16s  
151 files: 0.288s  
501 files: 0.959s

Honest scope: this is a controlled regression fixture (tests/fixtures.py), not a general detection rate. Real-repository accuracy is the next measurement. Source: [outputs/benchmark-results/benchmark.json](#).

# Built today, planned next

## Implemented

FastAPI + SSE, Next.js graph UI  
Tree-sitter, Python AST, NetworkX  
YAML rules, SQLite cache  
LLM via Openrouter (openai/gpt-4o-mini)  
Markdown + SARIF reports

## Limits (stated)

Deep analysis: Python and JavaScript  
Sandbox: copied workspace + timeout, not a container  
Risk weights: prototype policy  
Accuracy on real repos: not yet measured

## Planned

GitHub Action / PR comment bot  
Semgrep, Gitleaks, OSV integrations  
Container sandbox  
More languages  
Weights calibrated on incident data

# Know what will break, why, and whether the fix holds

- Developers: instant, evidence-backed feedback.
- Reviewers: focus on design, not tracing callers by hand.
- Security teams: SARIF output and an audit trail per finding.
- Release managers: one clear go or no-go with reasons.

Live app: [perceptor-ai-release.vercel.app](https://perceptor-ai-release.vercel.app) | [github.com/Itx-Ace7/PERCEPTOR-ai](https://github.com/Itx-Ace7/PERCEPTOR-ai)

## Thank you. Questions?